Bloque 2. Seguridad, autenticación y sesiones

La seguridad de un servidor de aplicaciones es fundamental para proteger la información, controlar el acceso a los recursos y garantizar el correcto funcionamiento de las aplicaciones web.

En este bloque se estudian los mecanismos de autenticación y autorización proporcionados por Apache Tomcat, la administración de sesiones HTTP y las principales medidas de seguridad que deben aplicarse durante la configuración y administración del servidor.

1. Autenticación y control de acceso

1.1. Identificación, autenticación y autorización

En una aplicación web es necesario controlar quién puede acceder a los recursos y qué operaciones puede realizar. Para ello se utilizan diferentes mecanismos de seguridad.

Es importante distinguir tres conceptos:

- **Identificación:** proceso mediante el cual un usuario indica quién es, normalmente proporcionando un nombre de usuario.
- **Autenticación:** proceso que permite comprobar que el usuario es realmente quien dice ser. Puede realizarse mediante contraseñas, certificados digitales u otros mecanismos.
- **Autorización:** proceso que determina qué recursos y operaciones tiene permitido utilizar un usuario una vez autenticado.

> **Ejemplo: acceso a una aplicación de gestión**
>
> Una aplicación de gestión dispone de usuarios con diferentes responsabilidades:
>
> - Un administrador puede gestionar usuarios y modificar información.
> - Un empleado puede consultar y actualizar determinados registros.
> - Un visitante solamente puede consultar información pública.
>
> El usuario primero se identifica y autentica. Después, la aplicación comprueba sus permisos para decidir a qué funcionalidades puede acceder.

1.2. Usuarios, roles y permisos

Los sistemas de control de acceso suelen organizar los permisos mediante usuarios y roles.

- **Usuario:** representa una identidad que puede acceder al sistema.
- **Rol:** representa un conjunto de responsabilidades o permisos.
- **Permiso:** determina qué operaciones puede realizar una identidad sobre un recurso.

En lugar de asignar individualmente todos los permisos a cada usuario, se pueden crear roles y asociarlos a diferentes usuarios.

Usuario	Rol	Permisos
ana	administrador	Gestionar usuarios, consultar y modificar información.
pedro	empleado	Consultar y actualizar registros.
invitado	visitante	Consultar información pública.

1.3. Dominios de seguridad

Un dominio de seguridad es el entorno encargado de gestionar las identidades y los mecanismos de autenticación de una aplicación o servidor.

En Tomcat, los **Realms** proporcionan mecanismos para autenticar usuarios y obtener información sobre sus roles. La aplicación puede utilizar estos datos para controlar el acceso a sus recursos.

Tomcat dispone de diferentes implementaciones de Realm:

Tipo de Realm	Funcionamiento
MemoryRealm	Obtiene usuarios y roles desde un archivo XML cargado en memoria.
JDBCRealm	Obtiene la información de usuarios y roles desde una base de datos.
DataSourceRealm	Utiliza un DataSource para consultar usuarios y roles en una base de datos.
JAASRealm	Utiliza el sistema de autenticación y autorización JAAS de Java.

La elección del Realm depende de dónde se almacenen las identidades y de las necesidades de seguridad de la aplicación.

1.4. Configuración de usuarios y roles en Tomcat

Tomcat permite definir usuarios y roles mediante el archivo `tomcat-users.xml`. Su ubicación habitual es el directorio `conf/` de la instalación.

Este mecanismo puede utilizarse con un MemoryRealm para almacenar las identidades en un archivo XML.

Ejemplo: definición de usuarios y roles

```
<tomcat-users>

    <role rolename="manager-gui"/>
    <role rolename="admin-gui"/>
    <role rolename="empleado"/>

    <user username="admin"
          password="ClaveSegura123"
          roles="manager-gui,admin-gui"/>

    <user username="usuario1"
          password="Usuario123"
          roles="empleado"/>

</tomcat-users>
```

En este ejemplo se crean tres roles y dos usuarios. El usuario `admin` dispone de permisos de administración y `usuario1` tiene el rol de empleado.

Importante: este ejemplo utiliza contraseñas en texto claro únicamente para mostrar la estructura del archivo. En un entorno real deben utilizarse mecanismos de almacenamiento seguro de credenciales y evitar contraseñas débiles.

1.5. Mecanismos de autenticación HTTP

Tomcat permite configurar diferentes mecanismos de autenticación para proteger los recursos de una aplicación.

Autenticación BASIC

El mecanismo BASIC solicita al usuario un nombre de usuario y una contraseña mediante un cuadro de diálogo del navegador.

Las credenciales se transmiten codificadas mediante Base64, pero Base64 no es un mecanismo de cifrado. Por tanto, este sistema debe utilizarse siempre sobre una conexión HTTPS.

Ejemplo: configuración de autenticación BASIC

En el archivo `WEB-INF/web.xml` se puede establecer una restricción de seguridad:

```
<security-constraint>

    <web-resource-collection>
        <web-resource-name>Zona privada</web-resource-name>
        <url-pattern>/privado/*</url-pattern>
    </web-resource-collection>

    <auth-constraint>
        <role-name>empleado</role-name>
    </auth-constraint>

</security-constraint>

<login-config>
    <auth-method>BASIC</auth-method>
    <realm-name>Aplicacion Empresa</realm-name>
</login-config>

<security-role>
    <role-name>empleado</role-name>
</security-role>
```

Con esta configuración, las URL que comienzan por `/privado/` quedan protegidas y solamente pueden ser utilizadas por usuarios autenticados que tengan el rol `empleado`.

Autenticación FORM

La autenticación FORM permite utilizar páginas HTML propias para solicitar las credenciales del usuario.

El desarrollador puede personalizar la apariencia del formulario y las páginas que se muestran cuando la autenticación se realiza correctamente o falla.

Ejemplo: configuración FORM

```
<login-config>

    <auth-method>FORM</auth-method>

    <form-login-config>
        <form-login-page>/login.html</form-login-page>
        <form-error-page>/error-login.html</form-error-page>
    </form-login-config>

</login-config>
```

Las páginas de acceso y error deben respetar el protocolo de autenticación FORM de la especificación Servlet.

Autenticación mediante certificados

También es posible autenticar usuarios mediante certificados digitales. En este caso, el servidor comprueba la identidad a partir de un certificado presentado por el cliente.

Este mecanismo puede utilizarse en entornos que requieren un nivel elevado de seguridad o una identificación basada en certificados emitidos por una autoridad de confianza.

1.6. Restricciones de acceso mediante web.xml

El descriptor de despliegue permite establecer restricciones sobre determinadas rutas de una aplicación.

Mediante `security-constraint` se definen los recursos protegidos y los roles autorizados para acceder a ellos.

Ejemplo: proteger una zona de administración

```
<security-constraint>

    <web-resource-collection>
        <web-resource-name>Administracion</web-resource-name>
        <url-pattern>/admin/*</url-pattern>
    </web-resource-collection>

    <auth-constraint>
        <role-name>administrador</role-name>
    </auth-constraint>

</security-constraint>

<security-role>
    <role-name>administrador</role-name>
</security-role>
```

En este caso, solamente los usuarios que pertenezcan al rol `administrador` podrán acceder a las rutas protegidas.

2. Administración de sesiones HTTP

2.1. El protocolo HTTP y la falta de estado

HTTP es un protocolo sin estado (stateless). Esto significa que cada petición se procesa de forma independiente y el protocolo no mantiene automáticamente información sobre las peticiones anteriores.

Por ejemplo, cuando un usuario inicia sesión en una aplicación, el servidor necesita algún mecanismo que permita reconocerlo cuando realice nuevas peticiones.

Para resolver este problema se utilizan las sesiones y otros mecanismos de persistencia de información.

2.2. ¿Qué es una sesión HTTP?

Una sesión HTTP permite mantener información asociada a un usuario durante diferentes peticiones realizadas a una aplicación web.

El servidor genera un identificador de sesión que permite relacionar las peticiones posteriores con la información almacenada.

Una sesión puede utilizarse para:

- Mantener el estado de autenticación de un usuario.
- Almacenar los productos de un carrito de compra.
- Conservar preferencias de navegación.
- Guardar información temporal durante un proceso.
- Controlar el tiempo de actividad de un usuario.

2.3. Identificador de sesión y cookies

Una cookie es un pequeño dato que el servidor puede enviar al navegador para que este lo almacene y lo devuelva en futuras peticiones.

Tomcat utiliza habitualmente una cookie denominada `JSESSIONID` para identificar las sesiones HTTP.

El funcionamiento habitual es el siguiente:

1. El navegador realiza una primera petición a la aplicación.
2. Tomcat crea una nueva sesión.
3. El servidor asigna un identificador único.
4. El navegador recibe una cookie con ese identificador.
5. En las siguientes peticiones, el navegador envía la cookie.
6. Tomcat identifica la sesión y recupera la información asociada.

Ejemplo: cookie JSESSIONID

Una respuesta HTTP puede incluir una cabecera similar a esta:

```
Set-Cookie: JSESSIONID=ABC123XYZ; Path=/mi-aplicacion; HttpOnly
```

En las siguientes peticiones, el navegador puede enviar:

```
Cookie: JSESSIONID=ABC123XYZ
```

El identificador permite al servidor recuperar los datos de la sesión correspondiente.

2.4. Administración de sesiones mediante HttpSession

La API Servlet proporciona la interfaz `HttpSession` para gestionar sesiones desde una aplicación Java.

Mediante esta interfaz es posible crear sesiones, almacenar datos, recuperar información y finalizar sesiones.

Método	Función
`getSession()`	Obtiene la sesión actual o crea una nueva.
`getSession(false)`	Obtiene la sesión actual sin crear una nueva si no existe.
`setAttribute()`	Almacena un objeto en la sesión.
`getAttribute()`	Recupera un objeto almacenado.
`removeAttribute()`	Elimina un atributo de la sesión.
`invalidate()`	Invalida la sesión y elimina sus atributos.
`getId()`	Devuelve el identificador de sesión.
`getCreationTime()`	Devuelve la fecha de creación de la sesión.
`getLastAccessedTime()`	Devuelve el instante de la última petición asociada.
`setMaxInactiveInterval()`	Establece el tiempo máximo de inactividad.

2.5. Crear y recuperar una sesión

Una aplicación puede obtener la sesión asociada a una petición mediante el método `getSession()`.

Ejemplo: almacenar información en una sesión

```
import jakarta.servlet.http.HttpSession;
import jakarta.servlet.http.HttpServletRequest;

HttpSession sesion = request.getSession();

sesion.setAttribute("usuario", "ana");
sesion.setAttribute("rol", "administrador");

String usuario = (String) sesion.getAttribute("usuario");

System.out.println("Usuario: " + usuario);
```

La información almacenada estará disponible en otras peticiones que pertenezcan a la misma sesión.

2.6. Recuperar una sesión sin crearla

En determinadas situaciones interesa comprobar si existe una sesión antes de utilizarla. Para ello se puede emplear `getSession(false)`.

Ejemplo: comprobar si un usuario tiene una sesión activa

```
HttpSession sesion = request.getSession(false);

if (sesion != null) {

    String usuario =
        (String) sesion.getAttribute("usuario");

    if (usuario != null) {
        System.out.println("Usuario autenticado: " + usuario);
    }

} else {
    System.out.println("No existe una sesión activa");
}
```

2.7. Finalización e invalidación de sesiones

Cuando un usuario cierra sesión, la aplicación debe invalidar la sesión para evitar que se pueda seguir utilizando su información.

```
HttpSession sesion = request.getSession(false);

if (sesion != null) {
    sesion.invalidate();
}
```

El método `invalidate()` invalida la sesión y elimina los atributos asociados. Las peticiones posteriores no podrán utilizar esa sesión.

2.8. Caducidad de las sesiones

Las sesiones no deberían permanecer activas indefinidamente. Por motivos de seguridad y gestión de recursos, es necesario establecer un tiempo máximo de inactividad.

Este tiempo puede configurarse en el descriptor de despliegue `web.xml`.

Ejemplo: tiempo máximo de inactividad

```
<session-config>
    <session-timeout>20</session-timeout>
</session-config>
```

El valor se expresa en minutos. En este ejemplo, la sesión caducará después de 20 minutos de inactividad.

También puede configurarse desde el código Java:

```
HttpSession sesion = request.getSession();

sesion.setMaxInactiveInterval(600);
```

En este caso, el tiempo se expresa en segundos y la sesión caducará después de 600 segundos de inactividad.

2.9. Seguridad de las sesiones

Los identificadores de sesión deben protegerse porque permiten relacionar las peticiones con la identidad y los datos del usuario. Si un atacante consigue obtener un identificador válido, podría intentar utilizar la sesión de otra persona.

Entre las medidas de protección destacan:

- Utilizar HTTPS para proteger las comunicaciones.
- Activar la opción `HttpOnly` en las cookies de sesión.
- Configurar el atributo `Secure` para transmitir cookies únicamente mediante HTTPS.
- Utilizar el atributo `SameSite` para limitar el envío de cookies en contextos externos.
- Regenerar el identificador de sesión después de la autenticación.
- Invalidar la sesión al cerrar sesión.
- Establecer tiempos de caducidad adecuados.

> **Fijación de sesión (Session Fixation):** es un ataque en el que un atacante intenta conseguir que una víctima utilice un identificador de sesión conocido previamente por el atacante. Cambiar el identificador después de una autenticación satisfactoria ayuda a prevenir este problema.

3. Seguridad del servidor de aplicaciones

3.1. Principios básicos de seguridad

La seguridad de un servidor de aplicaciones debe abordarse desde diferentes niveles: sistema operativo, servidor, aplicación, red y almacenamiento de información.

Los principales principios que deben aplicarse son:

- **Confidencialidad:** garantizar que solamente las personas autorizadas puedan consultar la información.
- **Integridad:** evitar modificaciones no autorizadas de los datos.
- **Disponibilidad:** garantizar que los servicios estén disponibles cuando sean necesarios.
- **Mínimo privilegio:** proporcionar a cada usuario y servicio únicamente los permisos imprescindibles.

3.2. Protección de credenciales

Las credenciales de acceso son uno de los elementos más sensibles de cualquier sistema. Una exposición de contraseñas puede permitir que usuarios no autorizados accedan al servidor o a las aplicaciones.

Algunas recomendaciones son:

- Utilizar contraseñas largas y difíciles de adivinar.
- No utilizar la misma contraseña en diferentes servicios.
- Evitar almacenar contraseñas en texto claro.
- Utilizar mecanismos de hash adecuados para las contraseñas.
- Limitar el acceso a los archivos que contienen credenciales.
- No publicar credenciales en repositorios de código.
- Utilizar variables de entorno o sistemas de gestión de secretos cuando proceda.

> **Importante:** codificar una contraseña mediante Base64 no la protege. Las credenciales deben transmitirse mediante conexiones cifradas y almacenarse mediante mecanismos diseñados específicamente para ello.

3.3. Permisos del sistema de archivos

Tomcat necesita acceder a sus archivos de configuración, bibliotecas, aplicaciones y registros. Sin embargo, el proceso del servidor no debería ejecutarse con privilegios administrativos innecesarios.

En Debian, la instalación mediante paquetes suele utilizar un usuario específico para ejecutar Tomcat.

Podemos consultar el usuario del proceso mediante:

```
ps aux | grep tomcat
```

También podemos consultar los permisos de un directorio:

```
ls -ld /var/lib/tomcat10
ls -ld /etc/tomcat10
ls -ld /var/log/tomcat10
```

Los permisos deben configurarse para que el usuario del servicio pueda acceder a los recursos necesarios, sin permitir modificaciones innecesarias.

Ejemplo: principio de mínimo privilegio

Si Tomcat necesita leer una configuración, no es necesario que todos los usuarios del sistema puedan modificarla.

```
sudo chown root:tomcat /etc/tomcat10
sudo chmod 750 /etc/tomcat10
```

Este ejemplo ilustra la asignación de permisos restrictivos. Antes de modificar permisos en una instalación real debe comprobarse el usuario y grupo que utiliza el servicio.

3.4. Configuración segura de Tomcat

Una instalación predeterminada no siempre ofrece el nivel de seguridad necesario para un entorno de producción. Es importante revisar la configuración y desactivar las funcionalidades que no se utilicen.

Algunas medidas recomendadas son:

- Eliminar aplicaciones de ejemplo que no sean necesarias.
- Restringir el acceso a Manager App y Host Manager.
- Utilizar usuarios y contraseñas específicos para la administración.
- Desactivar conectores y servicios que no se utilicen.
- Evitar ejecutar Tomcat como usuario root.
- Mantener Java y Tomcat actualizados.
- Configurar correctamente los conectores HTTP y HTTPS.
- Limitar las conexiones entrantes mediante el cortafuegos.

3.5. Configuración de conexiones seguras mediante HTTPS

HTTPS utiliza TLS para cifrar la comunicación entre el navegador y el servidor. Esto permite proteger la confidencialidad y la integridad de los datos transmitidos.

Para utilizar HTTPS es necesario disponer de un certificado digital y configurar un conector seguro en Tomcat.

Ejemplo: conector HTTPS en Tomcat

```
<Connector
    port="8443"
    protocol="org.apache.coyote.http11.Http11NioProtocol"
    SSLEnabled="true">

    <SSLHostConfig>
        <Certificate
            certificateKeystoreFile="/etc/tomcat10/servidor.p12"
            certificateKeystorePassword="clave"
            certificateKeystoreType="PKCS12" />
    </SSLHostConfig>

</Connector>
```

Este ejemplo muestra una configuración básica de un conector HTTPS utilizando un almacén PKCS12. La ruta y las credenciales deben adaptarse al entorno.

3.6. Protección de aplicaciones de administración

Tomcat incorpora herramientas web que permiten administrar el servidor. Entre ellas se encuentran Manager App y Host Manager.

Estas herramientas pueden realizar operaciones sensibles, por lo que su acceso debe estar limitado a usuarios autorizados.

No es recomendable exponer directamente estas interfaces a Internet. Deben utilizarse restricciones de acceso, contraseñas seguras y, cuando sea posible, redes administrativas separadas.

3.7. Registros y auditoría

Los registros permiten conocer las actividades realizadas en el servidor y facilitan la detección y resolución de incidentes.

Es recomendable registrar y revisar:

- Intentos de autenticación fallidos.
- Errores de ejecución de aplicaciones.
- Problemas de conexión.
- Accesos a recursos protegidos.

- Operaciones administrativas.
- Reinicios y paradas inesperadas del servicio.

En Debian, el servicio puede consultarse mediante `journalctl`:

```
sudo journalctl -u tomcat10

sudo journalctl -u tomcat10 --since "1 hour ago"

sudo journalctl -u tomcat10 -f
```

El último comando permite visualizar los nuevos registros a medida que se generan.

> **Auditoría:** registrar no significa almacenar indiscriminadamente toda la información. Los logs deben evitar incluir contraseñas, identificadores de sesión completos u otros datos sensibles. También es necesario controlar su acceso, conservación y rotación.

3.8. Desactivación de servicios innecesarios

Cada servicio o puerto abierto puede representar una posible superficie de ataque. Por ello, se deben mantener activos únicamente los componentes necesarios para el funcionamiento del sistema.

Podemos consultar los puertos en escucha mediante:

```
sudo ss -tulnp
```

También podemos consultar los servicios activos:

```
systemctl --type=service --state=running
```

Si un servicio no es necesario, se debe valorar su desactivación, teniendo en cuenta las dependencias del sistema.

3.9. Copias de seguridad y mantenimiento

La seguridad también implica garantizar que la aplicación pueda recuperarse ante errores, fallos del sistema o incidentes de seguridad.

Es recomendable mantener copias de seguridad de:

- Aplicaciones desplegadas y sus archivos WAR.
- Archivos de configuración de Tomcat.
- Certificados y almacenes de claves.
- Configuraciones de recursos y conexiones.
- Bases de datos utilizadas por las aplicaciones.

Las copias deben almacenarse de forma segura y comprobarse periódicamente mediante pruebas de restauración.

4. Ejercicios prácticos de aplicación

Ejercicio 1. Usuarios y roles en Tomcat

Configura un sistema de usuarios y roles para una aplicación web desplegada en Tomcat.

- Define diferentes roles de usuario.
- Configura usuarios asociados a esos roles.
- Establece restricciones de acceso a diferentes rutas.
- Configura un mecanismo de autenticación.
- Comprueba el acceso con usuarios que tengan diferentes permisos.

Ejercicio 2. Autenticación de una aplicación

Desarrolla una aplicación web con una zona pública y otra protegida.

- Configura autenticación BASIC o FORM.
- Protege una ruta mediante restricciones de seguridad.
- Establece roles con diferentes niveles de acceso.
- Comprueba el comportamiento con usuarios autorizados y no autorizados.

Ejercicio 3. Gestión de sesiones

Implementa una aplicación que permita iniciar y cerrar sesión, manteniendo información del usuario.

- Obtén una sesión mediante `HttpSession`.
- Almacena y recupera información del usuario.
- Comprueba el identificador de sesión.
- Configura un tiempo máximo de inactividad.

- Implementa el cierre de sesión mediante `invalidate()`.
- Comprueba el comportamiento después de la caducidad.

Ejercicio 4. Seguridad del servidor

Realiza una revisión de seguridad de una instalación de Tomcat en Debian.

- Identifica el usuario que ejecuta Tomcat.
- Comprueba los permisos de los directorios principales.
- Revisa las cuentas de administración.
- Comprueba los puertos abiertos.
- Localiza y analiza los registros del servicio.
- Identifica servicios o aplicaciones de ejemplo que no sean necesarios.
- Propón medidas de mejora de la configuración.

---

5. Resumen del bloque

- La autenticación permite comprobar la identidad de los usuarios, mientras que la autorización determina sus permisos.
- Los roles permiten agrupar permisos y asignarlos de forma organizada a diferentes usuarios.
- Tomcat utiliza Realms para obtener información de usuarios y roles y proporcionar mecanismos de autenticación.
- Los mecanismos BASIC y FORM permiten proteger recursos de las aplicaciones web mediante autenticación HTTP.
- Las sesiones HTTP permiten mantener información entre diferentes peticiones de un mismo usuario.
- HttpSession proporciona métodos para crear, consultar, modificar e invalidar sesiones.
- Las cookies permiten transportar el identificador de sesión entre el navegador y el servidor.
- Los tiempos de caducidad y la protección de las cookies son medidas fundamentales para la seguridad de las sesiones.
- La seguridad de Tomcat requiere proteger las credenciales, configurar correctamente los permisos y reducir los servicios innecesarios.
- Los registros, las actualizaciones y las copias de seguridad son elementos esenciales de la administración segura.